

Type System

is-function

Description: Checks if a type is a function type.

Parameters:

- Type: The type atom to be checked.

Return: `True` if the type is a function type, `False` otherwise.

Example:

```
!(is-function (-> Atom Atom)) ; Returns True
!(is-function Atom) ; Returns False
```

type-cast

Description: Attempts to cast an atom to a specific type within an atomspace context.

Parameters:

- Atom: The atom to cast.
- Type: The target type.
- Space: The atomspace to use as context.

Return: The atom if casting is successful, or an `Error` atom if not.

Example:

```
(: type1 Type)
!(type-cast A type1 &self) ; A
!(type-cast 1 type1 &self) ; Error 1 BasType
```

match-types

Description: Checks if two types can be unified and returns one value if so, another - otherwise.

Parameters:

- Type1: The first type.
- Type2: The second type.
- Then: Atom to be returned if types can be unified.
- Else: Atom to be returned if types cannot be unified.

Return: Third or fourth argument

Example:

```
!(match-types Atom Atom "Matched!" "Didn't match") ; Returns "Matched!"  
!(match-types Atom Number "Matched!" "Didn't match") ; Returns "Didn't match"
```

first-from-pair

Description: Gets a pair as a first argument and returns first atom from pair.

Parameters:

- Pair: Pair.

Return: First atom from a pair.

Example:

```
!(first-from-pair (A B)) ; Returns A
```

match-type-or

Description: Checks if two types can be unified and returns result of OR operation between first argument and type checking result.

Parameters:

- Folded: Boolean value

- Next: First type.
- Type: Second type.

Return: True or False

Example:

```
!(match-type-or True Number Number) ; Returns True  
!(match-type-or False Number Number) ; Returns True  
!(match-type-or True Number Bool) ; Returns True  
!(match-type-or False Number Bool) ; Returns False
```